Snog's Timer Manager

Snog's Timer Manager is a lightweight, flexible, and user-friendly timer system for Unity. Designed to be intuitive and efficient, it allows you to create, manage, and visualize timers with minimal setup — whether you're working in the editor or building for runtime.

This system is perfect for handling timed events, cooldowns, puzzle logic, animation triggers, or anything that needs reliable time control.

Features

- **Action-Based Callbacks**
  The manager now uses `System.Action` instead of `UnityEvent`, offering faster performance and cleaner code. You can still use UnityEvents in scenes through a helper component if needed.
- **Full Runtime Control**
  Pause, resume, stop, or remove timers individually or in bulk.
- **Editor Integration**
  The custom inspector displays active timers, smooth progress bars, and control buttons during play mode.
- **Lightweight and Clean API**
  Built for clarity and performance.

Table of Contents

Core Concepts

- **GameTimer**: The runtime class representing a single timer with duration, looping, and completion callback.
- **TimerManager**: Singleton that creates, updates, and controls all active timers.

Initial Setup Guide

1. Import the Asset

Drag the `Snog/TimerManager` folder into your Unity project.

2. Add the TimerManager to Your Scene

- Create an empty GameObject and add the `TimerManager` component.
- It automatically persists and manages timers across scenes.

3. You're Ready to Go!

Using Timers in Code

Create a Timer Manually

```
TimerManager.Instance.CreateTimer("MyTimer", 5f, false, () =>
Debug.Log("Timer done!"));
```

Pause, Resume, Stop or Remove Timers

```
TimerManager.Instance.PauseTimer("MyTimer");
TimerManager.Instance.ResumeTimer("MyTimer");
TimerManager.Instance.StopTimer("MyTimer");
TimerManager.Instance.RemoveTimer("MyTimer");
```

Global Control

```
TimerManager.Instance.PauseAllTimers();
TimerManager.Instance.ResumeAllTimers();
```

```
TimerManager.Instance.StopAllTimers();
```

FAQ

Q: Will this work in builds?

Yes! It will work on builds.

Q: Are UnityEvents supported?

Not directly — the system uses Actions for performance and simplicity.

Q: Can I add new timers at runtime?

Absolutely. Just call `CreateTimer()` with any ID and duration.

Q: Can I safely remove timers?

Yes, call `RemoveTimer("TimerID")` to remove it.

License